

Contract Playbook Comparison System

A retrieval-augmented system for automated contract clause review against a standard playbook

Team members

- Mannat — 1024240023
 - Upasna Wadhwa — 1024240022
 - Yuvicka — 1024240016
-

Course

Software Engineering — UCS503

Lab component

Requirements → Design → Implementation → Testing

Motivation

Why automated contract review is a real, unsolved workflow gap

100+

pages

Average length of vendor contracts legal teams review manually before signing.

Hours

per contract

Time spent by legal/procurement staff cross-checking each clause against internal policy.

Inconsistent

outcomes

Manual review quality varies by reviewer — critical deviations are frequently missed.

Industry relevance

Contract Lifecycle Management (CLM) is an established, funded software category — vendors such as Ironclad, LinkSquares, and Lawgeex sell automated contract-review tools to enterprises. This project builds a scoped, technically grounded version of that same real-world workflow, using established NLP and information-retrieval techniques rather than a toy use case.

Problem Statement

Defining the gap between the current (manual) process and the required system

Current system — limitations

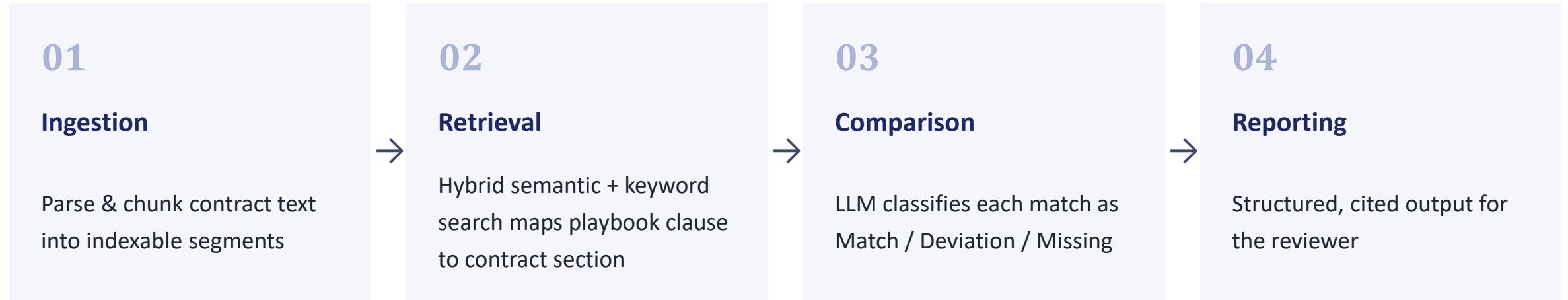
- No standardized process for clause-level comparison against internal policy
- Manual review does not scale with contract volume
- No audit trail linking a decision back to its exact source text
- Reviewer output is inconsistent and non-reproducible

Required system — objectives

-  Retrieve the contract section matching each playbook clause type
-  Classify each clause as Match, Deviation, or Missing
-  Cite the exact source location for every generated verdict
-  Maintain grounded, non-hallucinated outputs (verifiable against source)

Proposed Solution

A retrieval-augmented generation (RAG) pipeline, structured as independently testable modules



Software engineering approach

- Agile/Scrum: SRS → design → sprint-wise implementation → verification
- Modular architecture with defined component interfaces (pipeline stages)
- Unit + integration testing per module; validation against ground-truth dataset

Core stack

Python · Sentence-Transformers · FAISS / ChromaDB · LLM API · RAGAS (evaluation) · Streamlit (demo UI)